

Continuity Without Capability: Recurrent State, Exact Replay, and the Limits of a Proven Path

Flyxion
Independent Researcher

September 2026

Abstract

The Recurrent Looped Transformer (RLT) is most usefully read not merely as a novel architecture but as an operational baseline for replay-exact recurrent computation: a precise account of what must be preserved to reproduce one specified state transition, against which stronger claims about historical persistence, memory, and reasoning can be measured. Continuous descent, in which every token, prompt, response, tool return, and later turn, passes through the same recurrent decoder transition without an undeclared reset, is demonstrated exactly; faithful preservation, the survival of task-relevant distinctions across that descent, is not presumed, and this separation organizes what follows. Four propositions are routinely collapsed in discussions of recurrent and looped architectures: that a path exists, that influence survives along it, that the histories it distinguishes remain distinguishable, and that a capability results. RLT establishes the first exactly within its reference mathematical computation and leaves the remaining three open by design; what follows develops what would have to be true, and what would have to be measured, for the second, third, and fourth to hold, including a contraction criterion, a finite-state capacity bound, and a concrete probing protocol that constrain parts of this program analytically or experimentally without settling it outright. By formalizing the exact conditions required for current-policy replay, the paper exposes a common analytical shortcut: conflating structural continuity with cognitive capability. Its treatment of administrative versus constitutive boundaries is extended into a broader account of continuation in which some discontinuities are declared operators rather than accidents of implementation, with two complementary formal criteria: a persistent-noninterference condition for operators that must leave some projection of the state entirely untouched, and a write-isolation and read-transparency pair for operators whose purpose is to revise the protected content itself, illustrated with one possible state-level realization. RLT is neither an implementation of stronger memory architectures such as MEM|8 nor a competitor to them, and it is not the exclusive standard either; the essay treats RLT’s replay-exact recurrent state, MEM|8’s dynamic associative accessibility, and a history-first account of provenance and admissible continuation as three intersecting rather than nested standards, and separating their respective claims from one another matters as much as what is proved about any one of them.

1 The Path Is Real

Most discussion of recurrent and looped language models moves quickly to the interesting question, whether the recurrence produces better reasoning, and passes over a prior question that is rarely specified with any rigor: what, exactly, is being carried forward, and under what conditions is it the same thing being carried forward twice. Zhang’s Recurrent Looped Transformer (RLT) is notable for refusing to skip that step [1].

The architecture’s defining move is unglamorous by design. A causal Transformer encoder converts the token history, prompt, generated response, tool result, and any later turn, into key-value memory. A recurrent Transformer decoder then processes every one of those tokens through the same transition, merging each token’s encoder representation with the decoder’s own previous output. There is no separate mode for “the prompt” and “the response.” There is one state, updated once per token, for as long as the sequence continues.

This is a modest claim stated exactly. After t tokens have been processed, the state has traversed tL_D decoder blocks, where L_D is the decoder’s depth. The path has no fixed ceiling; it grows exactly as far as the sequence does, while each individual token still receives a fixed, bounded amount of computation. Zhang calls this “latent reasoning with infinite depth,” and the phrase is doing two jobs at once, one supportable and one not yet supportable. The path is real. Its length is unbounded. Both of these are proved. Whether reasoning travels along it is a separate matter, and the paper is careful, if not always in its titling then consistently in its qualifications, to say so.

The move itself is not new. Hybrid Transformer-recurrent decoding, encoder-derived reusable memory, and output-to-input latent feedback across tokens all have direct precedents, from early Transformer-RNN hybrids for translation through Feedback Transformer’s exposure of past representations to later computation [3], Universal Transformer’s depth-wise weight sharing [4], Block-Recurrent and Recurrent Memory Transformers’ persistent state vectors across segments [5, 6], and YOCO’s reusable encoder-derived key-value memory [7]. RLT’s originality, as the paper itself acknowledges, is compositional rather than elemental: it is the combination of full-decoder recurrence, exact replay semantics, and boundary invariance, argued together and proved rather than assumed, that distinguishes it from its precedents.

RLT is not simply a claim about intelligence deferred; it is an operational baseline for what any architecture must satisfy to reproduce a specified recurrent computation exactly, precise enough to expose exactly where the harder questions about memory, capability, and reasoning begin. That baseline is not, however, the only relevant standard for persistence in general; a system can satisfy RLT’s cache-validity conditions while failing stronger requirements concerning provenance, recorded refusal, or the preservation of collapsed distinctions, requirements that belong to a different, history-first account of what a state must answer for. The separation between what RLT proves and what it leaves open is worth as much as anything independently proved about any one of them.

2 What Counts as the State

The first discipline the paper imposes is definitional. It would be easy, and it is a common shortcut in looser recurrent-architecture writing, to say that the decoder’s final hidden vector is the memory, and be done with it. RLT does not permit this. The complete state is

$$H_t = (s_t, C_t^D),$$

where s_t is the decoder’s final output at position t and C_t^D is the retained key-value cache at every decoder layer, bounded by a sliding window W . Both components are causally load-bearing. Dropping C_t^D and keeping only s_t produces a state that looks complete but is not: it discards exactly the layer-specific activations that the decoder’s own attention mechanism depends on to reproduce later predictions.

The paper’s causality proof (its Proposition B.1, in the appendix) makes the same point from the other direction. H_t depends only on $x_{1:t}$ and the parameters, provided the encoder is causal, the memory is prefix-restricted, and the decoder’s sliding-window attention never reads a future position. This is a proof about the causal sufficiency of the specified state. The paper’s execution

equations and cache analysis further show that dropping a causally consulted component generally fails to reproduce the reference computation, although they do not establish that this representation is a mathematically minimal sufficient state. A cached prefix can stand in for forward replay only when it matches, exactly, the current parameters, the consumed token prefix, the position convention, the initialization, the window semantics, and the execution settings (Appendix C). Change any one of these and the cache is not a shortcut; it is a different, unvalidated claim about the system’s state.

This matters beyond RLT’s own architecture. It is a general lesson about what “the state of a system” means whenever that system’s behavior depends on more than what is visible in a transcript. A conversation’s text is a recoverable inscription. It is not, by itself, evidence of what the system that produced it has become through having processed it. Two systems could share an identical transcript and diverge entirely in their next output, if their H_t differ. RLT makes this point operationally rather than philosophically: it specifies, with the precision of an equation, exactly which quantities would have to match for the transcript-plus-parameters to be a sufficient description, and shows that a naive summary of “the hidden state” is not one of them.

This should not be read as showing that the transcript, once supplemented by the transition specification, simply is the history that matters. RLT’s reconstruction succeeds because its reference computation stipulates deterministic transitions, fixed parameters, fixed execution conventions, and a declared initial state; within that stipulation, transcript plus specification suffices to regenerate the one designated state H_t . A more demanding, history-first account would distinguish the visible transcript from a typed event history, and that history in turn from any derived state:

$$\text{transcript} \subsetneq \text{typed event history} \longrightarrow \text{derived state}.$$

The typed history can include provenance, refusal reasons, bindings, repair records, collapse rules, and execution events that are not recoverable from the visible text alone, and that a derived state may compress away entirely even when it remains adequate for RLT’s own purposes. In such an account, even transcript-plus-state may remain an incomplete record: the relevant history includes typed operations, provenance, rejected continuations, repair events, and the rules under which projections were formed. RLT avoids this larger problem rather than solving it, by fixing those rules externally and asking only what must be reconstructed to reproduce its reference transition. That is a legitimate and useful narrowing of scope; it should not be mistaken for having shown the narrower question is the only one worth asking.

3 Structural Depth and Effective Depth

Granting that the path exists and is exactly specified, the paper’s stronger claim, that this constitutes “latent reasoning,” requires several further propositions that are logically independent of the first and are not established by it. It is worth stating the chain explicitly, because collapsing it is the most common error in discussions of recurrent depth generally, not only in this paper:

path exists $\nRightarrow$ influence survives $\nRightarrow$ histories remain distinguishable $\nRightarrow$ capability emerges.

RLT proves the first link. The state composition

$$H_t = F_t \circ F_{t-1} \circ \cdots \circ F_1(H_0)$$

is a real, unbroken chain of t function applications, each one a full pass through L_D decoder blocks. No step in this composition is skipped, approximated, or reset at a serving boundary. That much is architecture, and it is proved rather than claimed.

The second link, that influence actually survives this composition rather than being contracted away, gated closed, or overwritten by recent input, is not established by the architecture and the paper does not claim it is. A composition of t functions can just as easily converge to a small family of attractor states, in which case the path is long but the information it carries about early tokens has effectively vanished. Nothing in the merge equation,

$$u_t = e_t + \alpha g_t \odot W_s r_{t-1},$$

rules this out. The gate g_t is learned; a learned gate can, in principle, learn to close, in the same way that gated recurrent architectures were originally designed to learn when to admit or block the flow of past information [17], and in the same way that a recurrent transition’s effective timescale can itself be a learned, input-dependent quantity rather than a fixed property of the architecture [18]. Whether RLT’s gate does close is an empirical question about a trained model, and RLT reports no trained model.

The third link is stronger still. Even if influence survives in some diffuse sense, detectable perhaps as a small perturbation in downstream logits, that is not the same as the system retaining distinguishable histories, histories that differ from each other in ways a downstream computation could exploit. A state that is sensitive to its input in an unstructured, noise-like way is not thereby a memory. Structured, task-relevant, history-dependent sensitivity is a much narrower and much more useful property, and it is the one that would actually license the word ”memory” or ”reasoning” for what is being carried along the path.

The fourth link, capability, is the one usually advertised and the one for which the least evidence has been assembled, both in this paper and in the recurrent-architecture literature generally. RLT reports no trained RLT model, no benchmark, no ablation, no scaling curve. This is not a defect in the paper on its own terms; the paper never claims otherwise. It is, however, the correct place to locate the gap between what has been proved and what has been suggested by the paper’s title and framing.

4 The Token as a Clock

RLT’s unbounded depth is not unbounded computation in the usual test-time sense. The architecture does not decide that a difficult problem deserves another recurrent step before producing an answer. Its temporal depth increases because another token has been consumed. Token position therefore serves as the model’s computational clock:

$$D(t) = tL_D.$$

A longer sequence produces a longer recurrent path whether or not the additional tokens are relevant, while a difficult problem expressed in a short sequence receives no additional recurrence unless the model first emits, receives, or internally represents more tokens.

This distinguishes RLT from architectures that allocate latent iterations independently of linguistic output, such as recurrent-depth approaches that iterate a block to scale test-time computation [9] or continuous-latent-space reasoning that inserts thought steps between ordinary tokens [8]. In a recurrent-depth model of that kind, one can in principle hold the visible context fixed and apply the transition repeatedly until a halting condition is met. In RLT, by contrast, recurrence is synchronized with movement through the token sequence. The model cannot remain at the same textual position while deepening its computation under the reference transition. Its path is indefinitely extensible, but the extension is purchased one consumed token at a time.

This matters because sequence length and problem difficulty are not equivalent quantities. A long conversational preamble may create thousands of recurrent transitions without requiring difficult inference. A compact mathematical question may require substantial computation after only a few tokens. The architecture guarantees that computation accumulates with history; it does not guarantee that computation is allocated according to need.

The distinction does not invalidate the phrase "unbounded temporal depth," but it narrows its interpretation. RLT has no fixed architectural bound on the depth of a state trajectory as the sequence grows. It does not yet possess an adaptive deliberation mechanism capable of increasing depth while holding linguistic time still. The token is both evidence presented to the system and the clock by which its recurrent computation advances. This places RLT within a broader recent family of proposals for cross-token latent feedback, including middle-layer temporal recurrence [13], gated fusion of the previous top-layer state with the next token embedding [14], source-layer state reuse through residual injection [11], layerwise persistent-key-value tiling [12], and latent-thought insertion during pretraining [10]. These proposals share the broader premise that computation can be extended through latent feedback, but they differ on the issue identified here: in RLT, ordinary token consumption paces recurrence, whereas latent-thought and recurrent-depth approaches, PonderLM-2's inserted latent positions and Coconut's dedicated latent-reasoning phase among them [10, 8], can introduce computational steps that are not identified with additional visible tokens at all.

This synchronization is not simply a constraint the architecture happens to inherit; it is closely connected to what makes the exactness results of later sections provable at all. A fixed, deterministic amount of work per consumed token, applied identically regardless of any adaptive halting decision, is exactly the property that lets Proposition 3.1's boundary invariance and the replay contract of Section 10 go through by a clean induction on token position. An architecture that instead allocated a variable number of recurrent steps per token, conditioned on some internal halting signal, would need that signal itself to be an exactly reproducible function of the parameters and history before the same replay guarantee could be recovered, and the guarantee would then depend on the halting mechanism rather than following directly from the transition's structure. The token-synchronized clock, in other words, is best read as the price RLT pays for provability rather than as an oversight relative to architectures that decouple depth from linguistic length. Whether that price is worth its computational cost, and whether an adaptive-depth architecture could recover an analogous exactness result through some other means, are separate and open questions; but within RLT's own design, the clock and the proof are the same choice.

5 Continuous Descent and Faithful Preservation

The word "continuity" can conceal two substantially different properties. The first is continuous descent: each state is produced from the preceding state by the declared transition, with no undeclared reset or substitution. RLT specifies this property exactly:

$$H_t = F_t(H_{t-1}).$$

Every state has a valid causal predecessor within the reference execution.

The second property is faithful preservation: distinctions present in an earlier history remain recoverable, discriminable, or behaviorally available after repeated transformations. Continuous descent does not entail faithful preservation. A sequence of perfectly valid transitions may contract many different histories into the same state:

$$H_j \neq H'_j \quad \text{but} \quad F_t \circ \dots \circ F_{j+1}(H_j) = F_t \circ \dots \circ F_{j+1}(H'_j).$$

Nothing discontinuous has occurred. No cache has been corrupted, no serving boundary has reset the system, and no execution rule has been violated. A distinction has nevertheless been lost. Classical results on gradient propagation in recurrent networks describe the same contraction from the training side: long products of Jacobians routinely vanish or explode, which is exactly the mechanism that would make such a collapse likely rather than merely conceivable [15, 16].

This is a more exact statement of the gap between structural and effective depth. Structural continuity concerns the ancestry of the state. Fidelity concerns the distinctions preserved through that ancestry. RLT proves that later states descend correctly from earlier ones within its reference computation. It does not prove that this descent remains injective, approximately invertible, or sufficiently discriminative over the histories relevant to a task:

$$\text{continuous descent} \not\Rightarrow \text{faithful preservation.}$$

Memory requires more than lawful descent. At minimum, it requires some controlled failure of convergence: histories that matter later must not all be mapped into an indistinguishable region before they are needed. The relevant question is therefore not merely whether the recurrent chain remains unbroken, but which distinctions survive its transformations, which are deliberately discarded, and whether the architecture has any means of knowing the difference.

Faithful preservation, in this sense, should not be read as demanding injective retention of the entire past; nothing this essay argues requires that standard, and no finite-state system could meet it indefinitely regardless of how it is designed. A more useful criterion is future-relative: preservation is faithful when a state retains the distinctions required to reach the futures a task or specification actually needs, together with enough provenance to repair or reinterpret the projection when those requirements change. That is a standard of admissibility relative to declared purposes, not a standard of archival completeness, and it is the standard the contraction criterion and capacity bound below are best read against: not "does the state remember everything," which nothing can, but "does it retain what a specified continuation set requires."

6 A Contraction Criterion for the Persistent Channel

The second link in Section 3's chain, that influence surviving a long recurrent path is not guaranteed by the path's existence, was stated qualitatively above. It can be made a good deal more precise for the one channel in RLT's state that is genuinely unbounded in extent: the persistent vector s_t , considered in isolation from the sliding-window cache C_t^D and the encoder memory $M_{\leq t}$. Isolating this channel is not an arbitrary simplification; it is the channel that must carry any decoder-side distinction once that distinction has aged out of the sliding window, since C_t^D forgets by hard eviction after W positions and $M_{\leq t}$, being encoder-derived, never contained the distinction to begin with. What follows is a sufficient condition, stated in terms of quantities the architecture already defines, under which this channel is provably contractive: it forgets geometrically fast regardless of how long the surrounding path has become.

Consider the direct dependence of s_t on s_{t-1} along the path running through the merge and then through the decoder stack, holding C_{t-1}^D , $M_{\leq t}$, and the encoder representation e_t fixed as this order of approximation isolates one specific channel of the total dependence: the total derivative $\partial s_t / \partial s_{t-1}$ also includes a route through C_{t-1}^D , since the cache and the persistent vector are jointly generated at every step, and that route is deliberately excluded here rather than analyzed. Writing $r_{t-1} = \text{RMSNorm}_s(s_{t-1})$ as in the merge equations, the chain rule gives the partial, cache-held-fixed

derivative

$$J_t^{(s)} := \left. \frac{\partial s_t}{\partial s_{t-1}} \right|_{C_{t-1}^D, M_{\leq t}, e_t \text{ fixed}} = \underbrace{\frac{\partial s_t}{\partial u_t}}_{J_D} \cdot \frac{\partial u_t}{\partial r_{t-1}} \cdot \frac{\partial r_{t-1}}{\partial s_{t-1}}.$$

The merge’s own Jacobian, from $u_t = e_t + \alpha g_t \odot W_s r_{t-1}$ with $g_t = \sigma(W_g[e_t; r_{t-1}] + b_g)$, has an exact form that includes the gate’s own dependence on r_{t-1} . Writing $W_{g,r}$ for the block of W_g that multiplies r_{t-1} ,

$$\frac{\partial u_t}{\partial r_{t-1}} = \alpha \left[\text{diag}(g_t) W_s + \text{diag}(W_s r_{t-1}) \text{diag}(g_t \odot (1 - g_t)) W_{g,r} \right].$$

Writing L_D for an operator-norm bound on the decoder stack’s Jacobian J_D , L_{norm} for the corresponding bound on RMSNorm’s Jacobian, and ρ_{merge} for an operator-norm bound on the full merge Jacobian above (both terms), all finite for any fixed, trained set of parameters, the operator norm of this partial, direct-path Jacobian satisfies

$$\|J_t^{(s)}\| \leq \rho := L_D \cdot \rho_{\text{merge}} \cdot L_{\text{norm}}.$$

This bound is on $J_t^{(s)}$, the direct recurrent-vector path alone; it is not a bound on the full Jacobian $J_t = \partial H_t / \partial H_{t-1}$, which also carries the route through the decoder-KV cache excluded above.

Proposition (contraction of the direct recurrent-vector path). If the corresponding exact one-step Jacobian norm is uniformly bounded by $\rho < 1$ over the trajectory, then the contribution transmitted exclusively through the direct recurrent-vector path, $s_{t-k} \rightarrow s_{t-k+1} \rightarrow \dots \rightarrow s_t$ without passing through the decoder’s own key-value cache, decays at most as ρ^k and therefore vanishes geometrically as $k \rightarrow \infty$. This does not establish contraction of the complete state transition, because influence may also propagate through the moving decoder-KV cache: an old cache entry can affect a newer activation, which enters a newer cache entry, which in turn affects a still-later activation after the original entry has been evicted. The cache has bounded instantaneous size but potentially unbounded relay depth, and the proposition above is silent about this second channel.

The proof is immediate composition along the isolated path: the one-step bound applies at every intermediate position, so k compositions bound the direct-path product by ρ^k . Two things are worth stressing about what this does and does not show. It does not show that RLT forgets, on either channel; ρ depends on trained quantities, g_t , W_s , $W_{g,r}$, and the decoder’s own Jacobian norm, none of which the architecture fixes in advance, and a trained model could in principle land on $\rho \geq 1$ even on the direct path alone, in which case this bound says nothing about whether influence survives there either. What the proposition does show is that “the path is long” and “the direct recurrent-vector path is contractive” are logically compatible, and compatible for an open, checkable set of parameter values; the paper’s own admission that gates and learned projections may suppress the practical contribution of long paths is not a rhetorical hedge but a description of exactly this regime, on at least this one channel. Measuring $\|g_t\|_\infty$, $\|W_s\|$, $\|W_{g,r}\|$, and an empirical operator-norm estimate of the decoder stack’s Jacobian gives a conservative sufficient bound on this channel alone, far cheaper than any task-level probe; a loose bound above one says almost nothing, and a bound below one establishes contraction only of the isolated direct-vector path, not of the complete state.

7 Behavioral Equality Is Not State Equality

Two recurrent states may produce the same next-token distribution without being the same state. Let the readout map be

$$p_t = R(H_t).$$

Then

$$R(H_t) = R(H'_t)$$

does not imply

$$H_t = H'_t.$$

The readout is a projection from a high-dimensional operative state into a probability distribution over the next token. Distinct states can therefore be observationally equivalent at one position while remaining dynamically distinct under later input. This is a familiar difficulty from the theory of state abstraction, where collapsing states that agree on immediate output can discard distinctions that later become behaviorally relevant [20].

This creates a problem for evaluating recurrent memory from output alone. Suppose two histories yield indistinguishable next-token probabilities immediately after a perturbation. It would be premature to conclude that the perturbation has been forgotten. A later token may interact differently with the two latent states and reveal a distinction that the earlier readout did not expose:

$$R(H_t) = R(H'_t), \quad R(F_{t+1}(H_t)) \neq R(F_{t+1}(H'_t)).$$

Conversely, a small immediate difference in logits does not establish durable memory. The difference may disappear at the next transition without ever becoming usable.

State comparison must therefore be counterfactual rather than merely instantaneous, in the spirit of predictive state representations, which define a state by the future observations it predicts rather than by its intrinsic content [19]. The relevant equivalence relation is not "produces the same output now," but "produces the same family of continuations under the relevant future inputs." For a set of admissible continuations $\mathcal{U}$, one could define states as behaviorally equivalent when

$$H \sim_{\mathcal{U}} H' \iff R(F_u(H)) = R(F_u(H')) \text{ for all } u \in \mathcal{U},$$

where F_u denotes the transition induced by continuation u .

This formulation also sharpens the meaning of capability. A latent distinction matters only if some relevant continuation can recruit it. Information that remains encoded but cannot affect any admissible future behavior is not operational memory for the system, whatever a probe trained outside the system might recover from its activations. What travels along the path must remain not only present, but available to the model's own subsequent dynamics.

8 External Tokens as Recurrent Computation

RLT applies the same state transition to assistant tokens, user messages, role delimiters, and tool results. From the standpoint of reinforcement learning, these token classes are not equivalent: only sampled policy actions receive importance-ratio factors. From the standpoint of recurrent state evolution, however, every consumed token performs computational work. An external token is simultaneously new evidence and another application of the decoder transition.

This produces an unusual coupling between interaction structure and computational depth. A tool call that returns a long document does more than add information to the encoder memory.

It advances the recurrent state through every token of the returned document. Two interfaces that provide semantically comparable evidence at different textual lengths can therefore induce different numbers of recurrent transformations before the next assistant action. Representation length becomes part of the execution policy.

The serving interface consequently participates in the model’s effective architecture even when it introduces no explicit reset. Tokenization, serialization format, role markers, tool verbosity, and the ordering of equivalent observations determine how many transitions occur and what intermediate states are produced. Proposition 3.1 establishes invariance to moving a boundary within a fixed token history. It does not establish invariance under alternative encodings of the same purported content, because alternative encodings are different histories.

This is not merely an efficiency concern. If recurrent computation is functionally important, verbose tool output may provide more transformation depth than concise output. If recurrence is contractive, the same verbosity may instead erode earlier distinctions before the model acts. An adversarially padded input could alter the model’s operative state even when its additional tokens contribute little propositional content. The architecture therefore makes serialization a causal variable.

A complete account of RLT capability would eventually have to distinguish sensitivity to information from sensitivity to presentation length. The appropriate test is not only whether the model uses external evidence, but whether semantically controlled variations in tokenization, verbosity, and ordering change its conclusions through their effect on recurrent depth. A continuous path makes every token part of the computation; it does not guarantee that every way of expressing the same evidence produces an equivalent computation.

9 A Capacity Bound on the Bounded Decoder State

The contraction criterion of Section 6 is parameter-dependent: whether it applies is a fact about a particular trained model, and a differently trained RLT instance could avoid it entirely. A complementary bound is available that holds for every instantiation of the architecture, trained or not, because it follows from dimension-counting alone rather than from any property of the learned weights. The bound below concerns only the bounded decoder state $H_t = (s_t, C_t^D)$, not RLT’s complete serving snapshot, which also includes the growing encoder memory $M_{\leq t}$ and encoder continuation cache; that distinction matters and is addressed directly below.

Fix a vocabulary of size V and suppose the persistent vector $s_t \in \mathbb{R}^d$ is realized at finite precision, so that each of its d coordinates takes one of at most N distinguishable values. The map from a token history to the resulting persistent vector,

$$x_{1:t} \longmapsto s_t(x_{1:t}; \Theta),$$

is, by the paper’s causality proof together with the transition’s determinism, a well-defined function of $x_{1:t}$ for fixed Θ . Its domain has size V^t , the number of possible length- t histories, while its codomain has size at most N^d , a quantity that does not grow with t at all. Once

$$t > t^* := \frac{d \log N}{\log V},$$

the domain strictly exceeds the codomain, and the map cannot be injective: there exist at least two distinct histories $x_{1:t} \neq x'_{1:t}$ with $s_t(x_{1:t}) = s_t(x'_{1:t})$.

The same argument extends to the complete decoder state $H_t = (s_t, C_t^D)$. Once $t \geq W$, the sliding-window cache C_t^D also has a size independent of t , bounded by the window width, the

per-layer key/value dimension, and the number of memory groups; call this bound N^{d_C} . The joint codomain size is then at most N^{d+d_C} , still a constant in t , so an analogous threshold $t^{**} \geq t^*$ exists beyond which H_t itself cannot injectively encode the full history either.

Proposition (bounded-decoder-state collision). For fixed architecture width d , window W , decoder depth, cache dimensions, and finite numerical precision N , there exists a finite threshold t^{**} , independent of the trained parameters Θ , such that for all $t > t^{**}$ the map $x_{1:t} \mapsto H_t = (s_t, C_t^D)$ is not injective.

This is a pigeonhole argument, not a claim about which histories collide or how much the collisions matter; it says nothing about whether the colliding histories are ever task-relevantly different, and the collisions may fall entirely among histories the model never needs to distinguish. Nor does it show that the bounded decoder state cannot be an unbounded-capacity record of the past in the sense of the full model: what it shows is narrower, that the bounded decoder state cannot be an injective record of arbitrarily long token histories. Two histories that collide in H_t can still produce different outputs downstream, because RLT’s complete serving snapshot also includes the encoder memory $M_{\leq t}$, which stores one key-value pair per token and so has size $O(t \cdot d_{KV})$, growing without bound alongside t . The proposition establishes a limit on decoder-derived historical compression, specifically, not a limit on the total distinguishability the served model can exhibit once encoder memory is consulted. Any decoder-side distinction, meaning any fact about how the decoder came to treat the history a particular way rather than merely which tokens occurred, that has aged out of C_t^D ’s window and is not independently recoverable by re-reading $M_{\leq t}$, is competing for space in a channel, s_t , whose total capacity is fixed once and for all by the architecture’s width, not by the length of the conversation it is asked to carry.

The threshold itself also needs a practical qualifier: t^{**} is a fact about the mathematical extension of the architecture to arbitrarily long sequences, not necessarily about any deployed context regime. For realistic d and precision N , t^{**} may vastly exceed every sequence length the model is ever asked to process, in which case the bound is true but practically vacuous for that deployment; the interesting empirical question is not whether some collision threshold exists, it must, but where it falls relative to sequence lengths actually in use, and how gracefully the decoder-side channel degrades as that threshold is approached.

10 Exact Replay, Inexact Claims

The paper’s strongest and most fully realized contribution is not in the architecture section but in its treatment of reinforcement learning, and it is worth dwelling on because it demonstrates the same discipline applied to a genuinely hard problem, with a genuinely correct answer.

The problem is this. A sampler generates a response under some behavior policy μ , recording each token’s log-probability under that policy. Later, after the model’s parameters have changed through an optimizer step, a trainer needs the current-policy log-probability of those same tokens, to compute an importance ratio

$$r_i(\Theta) = \exp(\log p_{\Theta}(y_i \mid c, y_{<i}) - \log \mu(y_i \mid c, y_{<i})).$$

The tempting shortcut is to reuse whatever hidden states the sampler already computed. RLT identifies exactly why this is wrong: those states were built under the old parameters. After an update, H_T under Θ_{new} is a different mathematical object from H_T under Θ_{old} , even for the identical token sequence, because every component of the recurrent transition, the encoder, the merge, every decoder block, depends on Θ . Reusing the old states does not evaluate the current

policy. When old parameter-dependent states are combined with updated weights, the result may correspond to neither the former policy nor the current one, but to an explicitly approximate hybrid computation. Exact current-policy evaluation requires reconstruction from the sequence beginning, or continuation from an exact prefix checkpoint produced under the same parameters and execution convention, for every optimization pass over a rollout.

This is a real result, stated with the same exactness as the causality proof, and it generalizes past RLT’s own architecture. Any policy whose forward computation is history-dependent in a way richer than a fixed-size prompt window faces this problem, and most treatments of recurrent or stateful RL replay in the wild do not treat it with this level of care. It is a sharper instance of the general behavior-policy-versus-target-policy distinction that off-policy evaluation has long depended on [22, 23], and it sits alongside the clipped-ratio surrogate objectives commonly used to tolerate ordinary policy lag [21], which RLT’s replay contract is careful to keep conceptually separate from architecture-induced state mismatch. Zhang’s own companion report develops the related kernel-level mismatch between prefill and decode execution that can silently violate this same replay contract [2]. Zhang’s contribution here is not the observation that stale states are wrong, which is close to obvious once stated, but the precise specification of what ”not stale” requires: exact parameters, exact prefix, exact positional and windowing conventions, exact initialization.

The contrast with the capability claims elsewhere in the paper is instructive rather than damning. Where the paper can prove something exactly, as with replay correctness or causal state-sufficiency, it does. Where it cannot, as with whether recurrent depth produces reasoning, it says so, repeatedly, in qualifying clauses that a less careful reader might skip past on the way to the abstract’s more exciting framing. The paper’s discipline is real; its title’s implicit promise slightly outruns it.

10.1 The Hybrid-Computation Bias: A Worked Example

The claim that reusing old states under new parameters produces an explicitly approximate hybrid computation, rather than either policy exactly, can be made concrete with a toy model small enough to compute by hand. Let the vocabulary be binary, $x_t \in \{0, 1\}$, and let a one-dimensional persistent state evolve by

$$s_t = \tanh(\theta s_{t-1} + e_t), \quad s_0 = 0,$$

with e_t a fixed scalar embedding of x_t and a scalar readout weight w producing $p_\Theta(x_{t+1} = 1 \mid x_{1:t}) = \sigma(w s_t)$ for parameters $\Theta = (\theta, w)$.

A sampler generates a rollout under $\Theta = (\theta, w)$, recording s_t^{old} at each step and the resulting behavior probabilities $\mu = \sigma(w s_t^{\text{old}})$. After an optimizer step, the parameters become $\Theta' = (\theta', w')$. Full replay, as the paper’s replay contract requires, recomputes

$$s_t^{\text{new}} = \tanh(\theta' s_{t-1}^{\text{new}} + e_t)$$

from $s_0 = 0$ under the new parameters throughout, giving the exact current-policy probability $p_{\Theta'} = \sigma(w' s_t^{\text{new}})$. The hybrid shortcut instead keeps the sampler’s old states and applies only the new readout, $\hat{p} = \sigma(w' s_t^{\text{old}})$, which is neither the behavior probability $\sigma(w s_t^{\text{old}})$ nor the current-policy probability $\sigma(w' s_t^{\text{new}})$.

The bias of this shortcut relative to the correct current-policy value is

$$\Delta_t = \sigma(w' s_t^{\text{old}}) - \sigma(w' s_t^{\text{new}}).$$

To first order in the parameter change $\delta\theta = \theta' - \theta$,

$$s_t^{\text{new}} \approx s_t^{\text{old}} + \left. \frac{\partial s_t}{\partial \theta} \right|_{\theta} \delta\theta, \quad \Delta_t \approx -\sigma'(w' s_t^{\text{old}}) w' \frac{\partial s_t}{\partial \theta} \delta\theta.$$

The sensitivity $\partial s_t / \partial \theta$ satisfies the recurrence

$$\frac{\partial s_t}{\partial \theta} = (1 - s_t^2) \left(s_{t-1} + \theta \frac{\partial s_{t-1}}{\partial \theta} \right),$$

which expands into a telescoping sum over every step of the replayed history,

$$\frac{\partial s_t}{\partial \theta} = \sum_{k=1}^t (1 - s_k^2) s_{k-1} \prod_{i=k+1}^t \theta (1 - s_i^2),$$

the same product-of-Jacobians structure that governs the contraction criterion of Section 6, here appearing on the parameter side rather than the state side. The bias Δ_t is therefore generically nonzero whenever this sensitivity is nonzero, which is generic, and the same recurrent sensitivity that allows parameter changes to alter long-range state evolution also creates a potential source of stale-state bias. The relationship is not an identity, however: its realized magnitude also depends on the size of the parameter update $\delta\theta$, on the readout’s local sensitivity $\sigma'(w' s_t^{\text{old}}) w'$, on the specific trajectory $\{s_k\}$, and, in an architecture with more than one channel, on which recurrent channel actually carries the task-relevant distinction; a small $\delta\theta$, an insensitive readout, or a task whose relevant information sits in a different channel can each make the bias small even when the sensitivity above is large. What the calculation rules out is only the stronger, and false, claim that a large recurrent sensitivity could coexist with a reliably small stale-state bias as a matter of architecture alone; the paper’s insistence on full reconstruction and this toy calculation are pointing at the same underlying quantity from two different directions, even though neither pins down the bias’s realized size without further information about the specific update and task.

11 The Cost of Continuity

None of this continuity is free, and the paper is honest about the bill even where it cannot yet pay it.

Encoder work parallelizes over known tokens, but the decoder does not. A prompt of length T requires T ordered decoder transitions, each containing L_D blocks, before generation can begin. Batching across independent sequences can improve hardware utilization, but it does not shorten this sequential chain for any single sequence. The paper gives its own arithmetic estimate for this cost:

$$O((L_E + L_D)(Td^2 + T^2d) + GTd^2 + L_D T \min(W, T)d).$$

The arithmetic work grows with context length, while the ordered decoder dependency constrains single-sequence latency regardless of encoder scheduling.

Training is more expensive still. Full backpropagation through time must preserve or recompute dependencies through s_t , every decoder-layer cache, and the encoder-side memory, across the entire sequence. The inference-time sliding window bounds what is retained for prediction; it does not bound what gradients must pass through during training, because those gradients still traverse computations that consumed KV entries later evicted from the inference cache. Checkpointing exchanges memory for recomputation. It does not shorten the dependency chain itself.

The reinforcement-learning case compounds this. Exact current-policy replay, as established in the previous section, requires reconstructing the full sequence from its start after every parameter update. Multiple optimization epochs over a rollout multiply this cost by the number of epochs. The paper defines the correct procedure without demonstrating that the correct procedure is affordable at any scale beyond the one it is content to leave unmeasured.

None of this is a flaw in the specification. It is the specification’s honest price, stated rather than hidden, and it is worth naming as a general property of any architecture that insists on continuity: continuity purchased at the level of exact state semantics is continuity purchased at the level of sequential compute, and the paper does not pretend otherwise.

12 Boundaries That Do Not Matter

The paper’s Proposition 3.1 is a small, clean result with a larger implication than its size suggests. Fix the parameters, the token sequence, the position convention, and the initial state. Then moving the point at which the system labels tokens “prompt” versus “response” does not change the next-token distribution, provided execution is deterministic and the encoder and decoder transitions are applied identically regardless of where that label falls. The proof is a straightforward induction: both scheduling choices, batched prefill followed by recurrent decoding, or fully incremental processing, start from the same H_0 and apply the same function at each step, so if their states agree at $t - 1$ they agree at t .

The implication is that “prompt” and “response” are serving-layer conveniences, not features of the underlying computation. They tell an inference server where to stop accepting input and start emitting output. They do not, and under this proposition cannot, silently reset or alter the transition law that governs the sequence. A boundary of this kind is administrative. It exists for the convenience of whoever is operating the system, not because the system itself treats that position in the sequence as special.

This is a useful and somewhat underappreciated distinction to make explicit, because a great deal of confused reasoning about “what an AI system remembers between messages” or “whether context resets at a new turn” trades on exactly this ambiguity. RLT’s proof draws the line cleanly for one specific architecture: if a boundary is purely a labeling convention, imposed by the serving stack rather than by the transition function, it should not change behavior, and if it does, that is a bug, not a feature of the model’s memory.

13 Boundaries That Perform Work

The proposition in the previous section proves less than it might first appear to, and the underclaim matters as much as the overclaim would have. Not every discontinuity in a persistent system is administrative. Some are the entire point.

Consider an operator that explicitly revises what a system is permitted to continue producing, rather than merely labeling where in a fixed sequence generation happens to stop. The clearest cases come from an existing operator algebra built around a structural option space Ω and an admissibility set $A(H)$ of continuations a history H currently permits. Pop commits to an event and removes the corresponding symbol from Ω itself, structurally consuming an option rather than merely narrowing which continuations are currently favored. Refuse does not remove an option from Ω ; it records that a proposed transformation is inadmissible, with a first-class reason, appending

$$\text{Refuse}(H, a, r) = H \frown \langle \text{refuse}, a, r \rangle, \quad \Omega(H') = \Omega(H), \quad A(H') \subseteq A(H),$$

so the refused branch remains historically present as evidence even as the admissible continuation set contracts. Bind declares a dependency or relation between elements without consuming any option, appending $\langle \text{bind}, u, v, \beta \rangle$ for a dependency type β ; it enriches the relational structure later continuations are evaluated against, and need not resolve an ambiguity or freeze a single value. Collapse forms an observation-relative quotient under a declared rule c , $O_c(H) = H / \sim_c$, removing

distinctions from the derived observable while the pre-collapse history, the equivalence rule, and an admissibility certificate for the collapse remain historically registered; what disappears is removed from the observable projection, not from the record. Repair does not overwrite an erroneous prior state; when two valid but contradictory paths are found, the governing procedure externalizes the resulting torsion into memory and preserves both paths until they can be scored, reconstructing a provenance-preserving history Γ^* from which admissibility $A(\Gamma^*)$ is evaluated only afterward, rather than pruning immediately. None of these five is a generic stand-in for the others: Pop commits and consumes an option, Refuse records inadmissibility while preserving the refused branch as evidence, Bind adds dependency without consuming options, Collapse forms a quotient while retaining what it collapsed, and Repair reconstructs a provenance-preserving history from which admissibility can again be evaluated. What all five share, and what a prompt/response label lacks, is that each is a declared transformation whose entire function is to change what the system's subsequent history is permitted to extend into. A boundary invariance proof that treated these the same way it treats a serving split, insisting they too must leave behavior unchanged, would not be strengthening the architecture's persistence; it would be disabling its capacity to ever revise itself.

The right general principle, then, is not that continuity means the absence of discontinuity. It is that continuity requires discontinuities to be declared by name, at the point where they are meant to operate, rather than smuggled in silently through an implementation detail that happens to fall at a convenient boundary. RLT's own transition function has no such declared operators; every token, without exception, receives the same update. This is precisely why its invariance proof can be stated and proved so cleanly: there is nothing in the architecture that is supposed to behave differently at any particular position, so proving that nothing accidentally does behave differently is a matter of induction, not of specifying which positions are exempt.

A system that does contain declared discontinuities, of the Pop/Refuse/Bind/Collapse/Repair kind, faces the harder and more interesting version of the same question: not "does an administrative boundary leave the transition law unchanged," which RLT answers, but "does an intentional boundary change exactly what it declares itself to change, and nothing else." RLT does not attempt this, and its clean proof depends on not attempting it. That is not a criticism of the paper; it is a note on the limit of what its proof technique can be asked to cover.

13.1 A Formal Criterion for Declared Operators

The distinction between administrative and constitutive boundaries can be given a precise form analogous to Proposition 3.1 and states, in state-transition terms, one property a declared operator would need to satisfy to preserve RLT's discipline while still doing real work. This is a formal criterion for a state-level operator in a general recurrent system, not a claim about what Pop, Refuse, Bind, Collapse, or Repair are; those remain history-level operations on (Ω, A, H) as characterized above, and the connection between the two levels is addressed after the constructions below.

Consider a general recurrent system whose complete state K_t evolves, by default, according to an ordinary transition Φ ,

$$K_t = \Phi(K_{t-1}, x_t; \Theta),$$

applied uniformly unless some externally triggered condition fires. Let $\mathcal{O} = \{O_1, \dots, O_m\}$ be a finite set of declared operators, each active only on its own disjoint trigger set $T_i \subset \{1, \dots, T\}$, with

$$K_t = \begin{cases} O_i(K_{t-1}, x_t) & t \in T_i \text{ for some } i, \\ \Phi(K_{t-1}, x_t; \Theta) & \text{otherwise.} \end{cases}$$

A first attempt at a locality condition, requiring only that an operator agree with Φ on some complementary projection ρ_i at the instant it fires, is not enough. Agreement at one step does not survive the next: an operator can alter its own declared component $\pi_i(K_t)$, and the very next application of Φ can mix π_i back into ρ_i , since an ordinary neural transition generally has no reason to keep its layers from mixing every state component together. Locality at a single instant is therefore not a claim about the future at all; what is needed is a condition on how the projection is permitted to evolve afterward, not merely on what the operator does when it fires.

Definition (persistent noninterference). An operator O_i is noninterfering with respect to a projection ρ_i if there exists a reduced transition $\bar{\Phi}_i$, depending only on $\rho_i(K)$ and x , such that

$$\rho_i(\Phi(K, x; \Theta)) = \bar{\Phi}_i(\rho_i(K), x; \Theta)$$

for every K and x , and

$$\rho_i(O_i(K, x)) = \bar{\Phi}_i(\rho_i(K), x; \Theta)$$

as well. The future evolution of ρ_i then depends only on ρ_i itself, never on π_i , and the declared operator agrees with the ordinary transition on this quotient at the moment it fires and at every later step, since both continue to evolve ρ_i by the same reduced rule $\bar{\Phi}_i$.

Proposition (projected invariance under a declared operator). Suppose two executions begin with equal ρ_i -projections and receive the same token sequence. One follows the ordinary transition Φ throughout; the other may replace Φ by O_i at positions in T_i . If Φ and O_i satisfy persistent noninterference with respect to ρ_i , then the two executions have identical ρ_i -projections at every position.

The proof is an induction on t that tracks $\rho_i(K_t)$ directly rather than the full state. At $t = 0$ the two executions agree by assumption. Suppose $\rho_i(K_{t-1})$ agrees across both. If $t \notin T_i$, both apply Φ , and noninterference's first clause gives $\rho_i(K_t) = \bar{\Phi}_i(\rho_i(K_{t-1}), x_t; \Theta)$ in either execution, so they agree. If $t \in T_i$, the operator-applying execution reaches $\rho_i(O_i(K_{t-1}, x_t)) = \bar{\Phi}_i(\rho_i(K_{t-1}), x_t; \Theta)$ by the definition's second clause, while the counterfactual all- Φ execution reaches $\bar{\Phi}_i(\rho_i(K_{t-1}), x_t; \Theta)$ by the first clause: the same value either way, so agreement is preserved. Because the reduced dynamics never reference π_i , this holds regardless of how $\pi_i(K_t)$ itself evolves or diverges between the two executions, which is exactly the property the earlier, instantaneous-locality attempt failed to secure.

Two scope limitations are worth stating rather than leaving implicit. First, this compares one execution that only ever applies Φ against one that substitutes O_i at T_i ; it says nothing yet about a system with several distinct declared operators $O_i, O_j, \dots$ all active in the same execution. Extending the result to that case requires noninterference of every operator with respect to every projection under comparison, not only O_i with ρ_i : one needs $\rho_i(O_j(K, x)) = \bar{\Phi}_i(\rho_i(K), x; \Theta)$ for every O_j permitted to occur alongside O_i , since noninterference of O_i with ρ_i says nothing on its own about what a different operator O_j might do to ρ_i . Second, the claim that administrative rescheduling in the sense of Proposition 3.1 is "recovered" at $\mathcal{O} = \emptyset$ overstates the connection: with no operators at all, the proposition above reduces to the truism that two identical executions remain identical, whereas Proposition 3.1 additionally compares two different execution schedules, batched causal prefill against fully incremental processing, and requires their equivalence as computational procedures. The result here is analogous to Proposition 3.1 in spirit, generalizing its concern with

boundaries that must leave computation unchanged, but it does not literally reduce to it without separately incorporating a model of scheduling into Φ itself.

This is a substantially stronger requirement than merely touching one slot of the state, and that is the point rather than a weakness in the formalization: it makes explicit that declared discontinuity is not secured by naming what an operator changes, but by architecturally protecting a quotient of the state so that its future evolution provably never depends on the declared component. This criterion suits operators whose entire purpose is to leave some designated projection exactly as untouched as an administrative boundary leaves the whole state in RLT itself, for instance a provenance tag or review flag that must never influence substantive downstream computation. Bind and Collapse, in their history-level sense above, belong to the opposite case, developed next as a state-level realization: their entire purpose is to revise the protected content itself, so the property they need is not that a projection stays unaffected, but a different, complementary pair of guarantees.

13.2 One Possible State-Level Realization: Block-Triangular Coupling

What follows is not a definition of Pop, Refuse, Bind, Collapse, or Repair; those remain history-level operations on (Ω, A, H) , as characterized above, not state coordinates. What follows is one possible mechanism for exposing selected consequences of such operations to a recurrent state while protecting their designated records from uncontrolled rewriting, in a system that must also maintain an ordinary working state alongside them. Persistent noninterference, as just defined, is the right protection for a declared operator that must leave some projection of the state completely unaffected. A register meant to hold the state-level trace of a Bind or Collapse decision is not of that kind: writing a dependency record or a committed observation is supposed to change the register, not leave it as the ordinary transition would have; demanding agreement with what the ordinary transition would have produced anyway would make the write indistinguishable from doing nothing. What such a register actually needs is a different pair of properties: a guarantee that it cannot be silently corrupted by the rest of the network’s ordinary mixing, and a guarantee that the rest of the network need not be given special-case logic to consume whatever it currently holds.

Split the complete state into two sectors, $K_t = (z_t, h_t)$: a designated quotient z_t , where bindings and collapsed choices live, and a working subsystem h_t , the general recurrent vector and cache that handle ongoing token processing. The default transition is block-triangular in one specific direction:

$$z_t = \bar{\Phi}(z_{t-1}, x_t), \quad h_t = \Psi(h_{t-1}, z_{t-1}, x_t).$$

The working subsystem may read the designated quotient, exactly as one would want, since the entire point of a binding is that later computation can consult it; but the designated quotient’s own update never takes h as an argument.

Definition (write-isolation). A quotient z is write-isolated if both the default transition and every declared operator acting on it are functions of (z, x) alone, never of h : $\bar{\Phi} = \bar{\Phi}(z, x)$ and $O_i = O_i(z, x)$ for every i .

Definition (read-transparency). The working subsystem’s transition is read-transparent with respect to z if $\Psi(h_{t-1}, z_{t-1}, x_t)$ is a single fixed function, applied identically regardless of whether z_{t-1} was produced by $\bar{\Phi}$ or by any declared operator O_i : h_t is a function of the current value of z_{t-1} , never of how that value was produced.

Proposition (uniform propagation). Under write-isolation and read-transparency, any two executions that agree on the resulting sequence $z_1, z_2, \dots$, whether that sequence is

produced by $\bar{\Phi}$ at every step, by declared operators at some steps, or by any mixture, and that apply the same Ψ , produce identical h_t at every step.

The proof is immediate induction: h_t depends on the previous state only through h_{t-1} , z_{t-1} , and x_t , via the single function Ψ , so agreement of both sequences up to $t - 1$ forces agreement at t . Write-isolation and read-transparency together give exactly what a substantive declared operator needs and nothing more: z cannot be corrupted by the working subsystem’s ordinary mixing, since h is not even an argument to its update, so a binding persists until $\bar{\Phi}$ or a further declared operator revises it; and the working subsystem requires no bespoke handling for post-operator states, since it consumes z through the same channel Ψ regardless of the operator’s involvement, meaning the operator’s effect propagates outward exactly as an ordinarily produced z value would. Nothing here claims h_t is unaffected by the operator, the opposite is the entire point, only that h ’s own transition rule is uniform and does not need to know that anything unusual happened.

With this structure in place, a simplified state-level analogue of each operator is direct to write down, though it should be read as a register-level stand-in for the history-level operations above, not as their definition: a register recording resolved dependencies or committed observations, rather than the general relational and quotient structures Bind and Collapse actually build. Writing z as containing a designated dependency slot and a hypothesis-weight subvector,

$$O_{\text{bind-register}}(z, x) = z \text{ with the dependency slot set to the relation determined by } x,$$

$$O_{\text{collapse-register}}(z, x) = \Pi(z),$$

where Π projects the hypothesis-weight subvector onto a single committed coordinate, zeroing the discarded alternatives in this projected register only, not in whatever history-level record of the collapse is maintained elsewhere. Persistence of the written value across further ordinary steps is a separate condition on $\bar{\Phi}$ itself, not automatic from write-isolation alone: $\bar{\Phi}$ must treat an already-written slot as a fixed point, $\bar{\Phi}(z, x) = z$ on that slot, for every x that is not itself a further declared revision, or the register’s content could be overwritten by the very rule meant to leave h out of the picture. Write-isolation guards against corruption from h ; it does not by itself guard against $\bar{\Phi}$ quietly drifting away from a committed value, which is a design requirement on $\bar{\Phi}$ that has to be verified independently.

This construction also makes precise why RLT cannot natively host a register of this kind. Its merge equation mixes the persistent vector densely: $u_t = e_t + \alpha g_t \odot W_s r_{t-1}$ has no zero block in W_s or in W_g carving out a subspace whose update ignores the rest of s_{t-1} , and every subsequent decoder block’s self-attention and FFN layers mix all coordinates of s_t and all cached entries in C_t^D together without restriction. There is no candidate z in RLT’s state whose update is write-isolated, because nothing in the specification zeroes out the couplings that would be required; achieving write-isolation would mean redesigning the merge and decoder-block weight matrices to be block-triangular in the sense above, a real architectural change, not a training outcome or a wrapper placed around the existing transition. What such a redesign would buy, even if carried out, is a state that can safely carry a trace of a Pop, Refuse, Bind, Collapse, or Repair event without that trace being overwritten by ordinary mixing; it would not, by itself, make the recurrent register the history-level record itself, which on the account above must also carry provenance, admissibility certificates, and the pre-collapse or pre-repair material that a fixed-width register cannot hold indefinitely regardless of how it is protected.

14 History Is Not Yet Memory

The remaining sections turn from close reading of RLT’s own proofs toward what those proofs suggest for broader accounts of persistent state; the register shifts accordingly, from verifying what the paper establishes to asking what its discipline would require of anything that claims to go further.

RLT’s relationship to richer proposals about machine memory, such as Chenoweth’s MEM|8 architecture [24], is easy to state wrongly in either direction. It would be wrong to say RLT implements MEM|8; it does not, and does not attempt to. It would also be wrong to dismiss the resemblance as superficial, because both architectures reject, for reasons that turn out to be structurally related, the idea that a system’s operative state is exhausted by its visible token history.

RLT establishes this rejection at the level of a proof. Two executions with identical visible text but different H_t need not represent the same policy state; text-only restoration requires recurrent replay to rebuild what the transcript alone does not contain. This is the same distinction, stated in an architecture’s own native vocabulary, between an inscription and the active consequences of having processed that inscription. MEM|8 proposes a considerably richer geometry for those consequences: memories represented as interference patterns in a dynamic wave grid, with natural temporal decay, emotional modulation, and cross-sensory binding, in which recall proceeds by cue-induced propagation, interference, and threshold-crossing resonance across hierarchical reactive memory layers, rather than by the recomputation of a fixed transition [24].

RLT contains none of that physics, and it does not need to in order to make its own point. Its state is an ordinary learned vector plus layerwise attention caches, evolved by Transformer blocks; nothing in the architecture corresponds to resonance or a decay law with an independently measurable time constant, and it has no analogue of the viscosity concept used in broader interpretive accounts of accessibility, propagation delay, and interference, which is not itself a stated variable of the cited MEM|8 architecture but belongs to the wider continuation framework such accounts are read alongside. Its forgetting is entirely implicit and, in one respect, structurally asymmetric: the sliding-window cache forgets by hard eviction after W positions, a forgetting rule stated with total precision, while s_t may carry an uncontrolled residual influence indefinitely, a retention rule stated with none. There is no explicit decay parameter for the persistent channel, no separation between fading and interference and reconstructive loss, only whatever the learned transition happens to do.

The lesson runs in both directions, which is what makes the comparison worth making rather than merely available to make. RLT’s insistence that a complete state must include every causally active channel, not only the one that is easiest to summarize, is exactly the standard a MEM|8-style account owes its own state description: not the currently dominant resonance alone, but every persistent field variable, retained interference structure, and admissibility-relevant residue needed to reproduce what happens next. And RLT’s cache-validity condition, that a cached state is valid only relative to the exact parameters and execution convention that produced it, is precisely the discipline a resonance-based trace also requires, restated in a different formalism: a field snapshot is not context-free either, and reusing one under changed filtering, stabilization, or decay parameters is the same category of error as reusing a hidden state under changed model weights.

The relationship is not one of strict containment, RLT inside MEM|8 inside some larger account of persistence, but closer to three intersecting standards addressing different questions. RLT asks whether a specified recurrent computation can be exactly reproduced from a cached state: replay-exact recurrent state. MEM|8 asks how stored residues become selectively, dynamically accessible again: associative accessibility and reconstruction. A history-first account, of the kind sketched in Section 9’s discussion of typed event histories, asks which historically grounded transformations,

Pop, Refuse, Bind, Collapse, Repair among them, preserve the futures that matter, and treats provenance and admissibility as first-class rather than derived. None of the three simply subsumes the others: RLT specifies, with a rigor MEM|8-style proposals do not yet match, exactly which channels are causally load-bearing and under what conditions a cached version of them remains valid, but it has no notion of provenance, refusal, or admissibility built into its state at all; MEM|8 proposes a richer accessibility structure than RLT’s but without RLT’s exactness about replay conditions; and a history-first account can retain provenance and admissibility that both the others compress away, at the cost of not specifying, the way RLT does, what a cached recurrent state must satisfy to be reused safely. What RLT leaves entirely open, on its own more modest terms, is the question of how a state should be structured so that some of what it carries is selectively, rather than merely uniformly, accessible later. The causal effects of history are permitted to propagate through RLT’s state. Whether the relevant history remains recoverable, distinguishable, or selectively accessible, in the stronger senses the other two standards ask about, is not guaranteed by RLT’s exactness and is not answered by appeal to it.

15 What Would Have to Travel

Taking the four-part chain from Section 3 seriously suggests a concrete, if demanding, empirical program, one the paper explicitly leaves for later work rather than omits by oversight. Two pieces of this program already admit partial analytic answers, rather than waiting on trained models at all: the contraction criterion of Section 6 turns the direct-path influence-survival question into a checkable inequality on trained quantities, and the capacity bound of Section 9 settles that some collision of distinct histories within the bounded decoder state must eventually occur regardless of training, though not necessarily within any sequence length actually deployed. What remains genuinely open, and does require the empirical work below, is measuring where a given trained instance falls relative to these bounds, and whether the collisions the capacity bound guarantees ever land on distinctions the task actually needs.

Establishing that influence survives, rather than merely that a path exists, invites two different kinds of evidence that should not be treated as interchangeable. The Jacobian

$$J_t = \frac{\partial H_t}{\partial H_{t-1}},$$

and its products over long histories, measured at a fixed, already-trained set of parameters, describe local stability: whether small perturbations to the state are amplified, damped, or preserved in magnitude as they propagate forward. The classical results on vanishing and exploding gradients [15, 16] are the right tool for understanding why such a product might behave badly during training, but they describe a difficulty in reaching a good set of parameters, not a property of the parameters a trained model actually lands on; a model that trains successfully despite these difficulties may still land on a transition whose Jacobian products are well-behaved along the directions that matter. What settles the question of survival, as used here, is closer to the predictive-state equivalence of Section 7: not whether a perturbation is locally amplified or damped in norm, but whether it changes the equivalence class $\sim_{\mathcal{U}}$ of the state under the continuations $\mathcal{U}$ that the task actually exercises. A perturbation with a large Jacobian norm that nonetheless leaves every task-relevant continuation’s output unchanged has not survived in the sense that matters; a perturbation with a small Jacobian norm that flips a downstream decision has. Jacobian spectra remain useful as a first, cheap diagnostic, and the perturbation experiments described next give the input-side complement to this state-side measure, but the equivalence-class question is the one a genuine survival claim needs to answer. Concretely, this still calls for perturbation experiments: alter an early token’s

encoder representation slightly and measure whether the perturbation is detectable in the output many tokens later, or whether it has been absorbed into an attractor indistinguishable from the unperturbed trajectory.

Establishing that histories remain distinguishable, rather than merely that influence survives in some diffuse sense, would require tasks explicitly designed to probe this: delayed-copy tasks, long-range causal retrieval, and controlled state-replacement experiments, in which the recurrent state is swapped for one produced by a different history while the visible token window is held constant. If predictions barely change under such a swap, the recurrent path is not doing the work its length suggests. If they change strongly but without discernible structure, the state is sensitive without being a memory in any useful sense. The result that would actually license the architecture’s stronger claims is structured, task-relevant, history-dependent sensitivity, neither of the two failure modes.

Establishing capability, finally, requires exactly the kind of evidence the paper does not provide and does not claim to: benchmarks, ablations that separate encoder-decoder division from parameter tying from sliding-window width from prompt-wide replay, and scaling curves showing that whatever is measured in the previous two stages actually improves with model size or training compute rather than remaining flat.

None of this is owed by the present paper on its own terms. A specification of architecture and execution semantics does not incur an obligation to have already run the experiments that would test claims it explicitly declines to make. The obligation arises at the next stage, whenever a trained RLT model’s advantages over conventional architectures are asserted rather than merely made structurally possible. Naming the program now, rather than after the fact, is simply a way of keeping the paper’s own discipline intact once the more exciting claims start to be made.

16 Probing Tasks for History Distinguishability: A Concrete Protocol

The preceding section named delayed-copy tasks and state-swap experiments as the right shape of evidence without specifying either in enough detail to run. What follows turns both into a single measurable protocol, built directly on the equivalence-class machinery of Section 7 and the two bounds of Sections 6 and 9, so that the empirical program and the analytic results answer the same question rather than sitting side by side unconnected.

16.1 The delayed-copy task

Fix a payload alphabet of size 2^k and insert a single marker token at position j carrying one payload value $p \in \{0, 1\}^k$, chosen uniformly at random. Follow it with L filler tokens, drawn from a distribution designed to be uninformative about p , and place a probe token at position $j + L + 1$ that requests the payload. Define the task’s distinguishability curve

$$D(L) = \text{TV}\left(R(H_{j+L+1} \mid p = 0), R(H_{j+L+1} \mid p = 1)\right),$$

the total variation distance between the readout distributions conditioned on the two payload values, averaged over fillers and, for $k > 1$, over payload pairs. $D(L) = 1$ means the payload is perfectly recoverable at gap L ; $D(L) = 0$ means it has become entirely unrecoverable from the readout at that position, whatever remains encoded elsewhere in H_{j+L+1} .

This curve is what the contraction criterion and the capacity bound each bear on, though neither predicts it outright, and the difference matters. In ordinary, unmodified RLT execution, the payload can influence H_{j+L+1} through three distinct causal routes: the direct recurrent-vector

path analyzed in Section 6, the moving decoder-KV relay that section explicitly excludes, and direct retrieval from the growing encoder memory $M_{\leq t}$, which never forgets the marker token at all. Only if encoder-memory access to the payload is masked after its initial presentation, decoder-cache mediation is disabled or separately intervened upon, and the readout is Lipschitz in s_t , does the isolated direct-path analysis yield a first-order diagnostic for an intervention-defined channel,

$$D_{\text{direct}}(L) \lesssim C\rho^L,$$

for some constant C depending on the payload’s initial imprint on s_j and the readout’s Lipschitz constant. This is an estimate of what the direct channel alone would show under that intervention, not a bound on the unmodified model’s ordinary delayed-copy performance, since the unmodified model retains the other two routes throughout.

The capacity bound requires the same care. It guarantees only that some distinct histories eventually collide in the bounded decoder state H_t ; it does not imply that this particular k -bit payload is among the distinctions discarded, nor that $D(L)$ must approach zero at any predictable point. A finite-state machine can preserve one selected k -bit statistic indefinitely while allowing a great many other histories to collide elsewhere; the pigeonhole argument is existential, not a claim about which distinctions a trained model chooses to keep. The two analytic results therefore constrain this experiment without directly predicting its curve: the direct-path bound implies geometric decay only for the isolated, intervened-upon channel, while the full, unmodified model may preserve the payload through decoder-KV relay or recover it outright from encoder memory; and the collision theorem establishes that H_t cannot injectively represent every possible history beyond a finite threshold, without implying that this payload is among what gets discarded. The experiment, run on the unmodified model, determines whether the trained system chooses to preserve the payload; it is not itself a direct verification of either bound.

The best-designed version of the experiment reports three curves rather than one: full-model $D(L)$ with every channel intact, $D(L)$ after masking encoder-memory access to the payload, and $D(L)$ after additionally intervening on decoder-cache relay so that only the direct recurrent-vector channel remains. The gaps between these three curves estimate how much of the full model’s distinguishability each channel is responsible for, and only the third curve is directly comparable to the weight-derived ρ from Section 6’s bound.

16.2 The state-swap experiment

The delayed-copy task measures whether $D(L)$ is nonzero; it does not, by itself, establish that the surviving distinction is task-relevant in the sense of Section 7’s equivalence classes rather than an artifact of the specific readout used. The state-swap experiment closes this gap. Given two full histories $x_{1:t}$ and $x'_{1:t}$ differing only before position j , produce their states H_t and H'_t , then evaluate both under a fixed, shared continuation u sampled from a task-relevant continuation set $\mathcal{U}$:

$$\delta(u) = \text{TV}(R(F_u(H_t)), R(F_u(H'_t))).$$

Averaging $\delta(u)$ over $u \sim \mathcal{U}$ gives an empirical, sampled proxy for whether $H_t \sim_{\mathcal{U}} H'_t$ in the sense defined earlier; exact verification of the equivalence relation over every continuation is not tractable, but a sampled estimate with enough draws from $\mathcal{U}$ bounds the probability that a task-relevant continuation would ever expose the difference. Two failure patterns are diagnostic rather than merely negative results. If $\delta(u) \approx 0$ for all sampled u despite $x_{1:t} \neq x'_{1:t}$ before j , the distinction has become behaviorally inert under every continuation the task exercises, consistent with either contraction or capacity saturation and distinguishable from them only by cross-referencing $D(L)$

from the delayed-copy task at the corresponding gap. If $\delta(u)$ is large but uncorrelated with any property of u that the task cares about, for instance if it fires on continuations that have nothing to do with the original distinction, the state is sensitive without being a memory in any useful sense: influence has survived structurally, in the sense of Section 6, without becoming a discriminable, useful memory in the sense of Section 7. Only a $\delta(u)$ that is both reliably nonzero and specifically correlated with the task-relevant content of u would license the stronger word.

16.3 What the protocol does and does not settle

Running this protocol across a range of L , payload sizes k , and continuation sets $\mathcal{U}$ would give an empirical map of exactly the second and third links in Section 3’s chain, calibrated against the two analytic bounds rather than reported as bare numbers. It would not, by itself, settle the fourth link. A model could exhibit a slowly decaying $D(L)$ and a well-correlated $\delta(u)$ on synthetic delayed-copy and swap tasks while still failing to recruit that same surviving distinction on a genuine reasoning task, since nothing in the protocol requires the task-relevant continuation set $\mathcal{U}$ used for validation to coincide with the continuations a real deployment actually samples. The protocol is deliberately scoped to the influence-survival and distinguishability links precisely because those are the links the analytic results above already speak to; capability remains, as before, a separate and larger undertaking that this protocol can motivate but not substitute for.

17 Continuity Without Capability

RLT’s contribution, read on its own terms, is an exact specification of a system’s persistent operative state, one that is careful never to be more than it is proved to be. It establishes that a computational path can extend continuously across an entire conversation, prompt, response, tool return, and subsequent turn, without a fixed depth ceiling and without administrative boundaries silently altering the transition law. It establishes, separately and just as carefully, that reusing this persistent state after a parameter update is not a shortcut but a category error, and that exact replay must reconstruct rather than reuse. What it does not establish, and does not claim to establish, is that anything traveling along the path it has built amounts to reasoning, memory in a stronger sense than mere carriage, or capability of any kind that would need to be benchmarked to be believed.

This is, if anything, the paper’s most valuable methodological contribution, more valuable than the architecture it happens to specify. It separates, cleanly enough to be stated as a chain of non-implications, four claims that recurrent and looped architectures generally invite readers to collapse into one: that a long computational path exists, that something survives passage along it, that what survives remains distinguishable rather than merged into indifference, and that the distinctions which remain participate in anything worth calling reasoning. RLT proves the first. It leaves the other three as an explicit, well-specified, and currently unanswered research program, rather than papering over them with a title that implies they have already been settled.

RLT therefore supplies something rarer than another claimed reasoning mechanism: a reference computation exact enough for failure to become measurable. Once continuity, replay, and boundary behavior are specified, contraction can be distinguished from persistence, sensitivity from memory, and memory from capability. The architecture does not close those distinctions. It makes it harder for subsequent work to evade them.

The state transition has been applied continuously, provably, without an undeclared reset at the level of the architecture’s own definitions. What remains unknown, and what the paper is honest enough to leave unknown, is whether anything capable of reasoning survives the journey.

References

- [1] Yifan Zhang. Recurrent Looped Transformer. Technical report, September 12, 2026. <https://github.com/yifanzhang-pro/recurrent-looped-tranformer>
- [2] Yifan Zhang et al. Reliable RL Scaling Requires Accounting for Prefill–Decode Kernel Mismatch. Technical report, Pretraining-RL-Science Project, August 2026. https://github.com/yifanzhang-pro/Pretraining-RL-Science/blob/master/Prefill_Decode_Kernel_Mismatch.pdf
- [3] Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. Addressing Some Limitations of Transformers with Feedback Memory. arXiv preprint arXiv:2002.09402, 2020. <https://arxiv.org/abs/2002.09402>
- [4] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In Proceedings of the 7th International Conference on Learning Representations, 2019. <https://arxiv.org/abs/1807.03819>
- [5] DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-Recurrent Transformers. In Advances in Neural Information Processing Systems, volume 35, 2022. <https://arxiv.org/abs/2203.07852>
- [6] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent Memory Transformer. arXiv preprint arXiv:2207.06881, 2022. <https://arxiv.org/abs/2207.06881>
- [7] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You Only Cache Once: Decoder–Decoder Architectures for Language Models. In Advances in Neural Information Processing Systems, volume 37, 2024. <https://arxiv.org/abs/2405.05254>
- [8] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. arXiv preprint arXiv:2412.06769, 2024. <https://arxiv.org/abs/2412.06769>
- [9] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling Up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. arXiv preprint arXiv:2502.05171, 2025. <https://arxiv.org/abs/2502.05171>
- [10] Boyi Zeng, He Li, Shixiang Song, Yixuan Wang, Ziwei He, Xinbing Wang, and Zhouhan Lin. PonderLM-2: Pretraining LLMs with Latent Thoughts in Continuous Space. arXiv preprint arXiv:2509.23184, 2025. <https://arxiv.org/abs/2509.23184>
- [11] Zeyi Huang, Xuehai He, Liliang Ren, Yiping Wang, Baolin Peng, Hao Cheng, Shuohang Wang, Pengcheng He, Jianfeng Gao, Yong Jae Lee, and Yelong Shen. Latent Recurrent Transformer: Architecture Exploration, Training Strategies, and Scaling Behavior. arXiv preprint arXiv:2605.26797, 2026. <https://arxiv.org/abs/2605.26797>
- [12] Costin-Andrei Oncescu, Depen Morwani, Samy Jelassi, Alexandru Meterez, Mujin Kwun, and Sham Kakade. The Recurrent Transformer: Greater Effective Depth and Efficient Decoding. arXiv preprint arXiv:2604.21215, 2026. <https://arxiv.org/abs/2604.21215>

- [13] Ziyang Cai, Xingyu Zhu, Yihe Dong, Yinghui He, and Sanjeev Arora. T2 MLR: Transformer with Temporal Middle-Layer Recurrence. arXiv preprint arXiv:2607.15178, 2026. <https://arxiv.org/abs/2607.15178>
- [14] Xi Wang, Ziyang Cai, Zheng Zhan, Harry Dong, Ying Fan, Gustavo de Rosa, Tim Pearce, and John Langford. Full-Bandwidth Transformer. arXiv preprint arXiv:2608.08888, 2026. <https://arxiv.org/abs/2608.08888>
- [15] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning Long-Term Dependencies with Gradient Descent Is Difficult. IEEE Transactions on Neural Networks, 5(2):157–166, 1994. doi:10.1109/72.279181
- [16] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the Difficulty of Training Recurrent Neural Networks. In Proceedings of the 30th International Conference on Machine Learning, pages 1310–1318, 2013. <https://arxiv.org/abs/1211.5063>
- [17] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. Neural Computation, 9(8):1735–1780, 1997. doi:10.1162/neco.1997.9.8.1735
- [18] Corentin Tallec and Yann Ollivier. Can Recurrent Neural Networks Warp Time? In Proceedings of the 6th International Conference on Learning Representations, 2018. <https://arxiv.org/abs/1804.11188>
- [19] Michael L. Littman, Richard S. Sutton, and Satinder Singh. Predictive Representations of State. In Advances in Neural Information Processing Systems, volume 14, 2001. <https://proceedings.neurips.cc/paper/2001/hash/1e4d36177d71bbb3558e43af9577d70e-Abstract.html>
- [20] Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a Unified Theory of State Abstraction for MDPs. In Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics, 2006. <https://arxiv.org/abs/2209.11963>
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>
- [22] Doina Precup, Richard S. Sutton, and Satinder Singh. Eligibility Traces for Off-Policy Policy Evaluation. In Proceedings of the Seventeenth International Conference on Machine Learning, pages 759–766, 2000.
- [23] Richard S. Sutton and Andrew G. Barto. Reinforcement Learning: An Introduction. MIT Press, Cambridge, Massachusetts, second edition, 2018. <http://incompleteideas.net/book/the-book-2nd.html>
- [24] Christopher Chenoweth and Alexandra Chenoweth. MEM|8: A Wave-Based Cognitive Architecture for Multimodal Memory Integration and Consciousness Simulation. Preprint, Zenodo, July 26, 2025. doi:10.5281/zenodo.16436298